

MASON MILLER

✉ masonmil@umich.edu  masonmil.com  linkedin.com/in/masonmil  github.com/masonmill

EDUCATION

University of Michigan

Bachelor of Science in Computer Science

Ann Arbor, MI

August 2022 – May 2026

- **GPA:** 3.9/4.0 | **Awards:** James B. Angell Scholar (×2), William J. Branstrom Prize
- **Relevant Courses:** Operating Systems, Advanced Operating Systems, Distributed Systems, Computer Architecture, Search Engine System Design
- **Technical Skills:** C++, C, Go, Bash | Linux, XNU, POSIX, pthreads, io_uring, SPDK, NVMe, TCP/IP, GDB, QEMU, Valgrind, ASan

PUBLICATIONS

DistDebug-Bench: Can LLM Agents Diagnose Bugs in Distributed Systems?

May 2026

- Yuxuan Jiang, Chenglin Liu, John Rose, **Mason Miller**, Ruthesh Thavamani, Juntao Wu, Yinwei Huang, Jiahao Xu, Prof. Ryan Huang
- Submitted to *NeurIPS '26*

Agentic Code-Level Debugging for Distributed Systems with Structured Hypothesis Exploration

April 2026

- Yuxuan Jiang, Yinwei Huang, Chenglin Liu, John Rose, Juntao Wu, **Mason Miller**, Ruthesh Thavamani, Prof. Ryan Huang
- Submitted to *NSDI '27*

EXPERIENCE

Citadel

Software Engineer Intern

New York, NY

June 2026 – August 2026

EECS Staff

Instructional Assistant

University of Michigan

August 2025 – Present

- **EECS 491: Distributed Systems (Winter 2026):** Covered replicated state machines, Paxos consensus, primary-backup replication, data consistency models, sharding, and scalability techniques
- **EECS 482: Operating Systems (Fall 2025):** Mentored students on threads, concurrency synchronization, CPU scheduling, virtual memory management, file systems, and distributed communication

Ordered Systems Lab

Research Assistant (Advisor: Prof. Ryan Huang)

University of Michigan

May 2024 – Present

- Supported research on Cirdan, an LLM distributed-systems debugger; built Docker-based testing infrastructure, curated evaluation benchmarks, and implemented context-injection pipelines to resolve structural ambiguity in complex telemetry
- Benchmarked asynchronous I/O libraries (libaio, liburing) with RocksDB, analyzing throughput, latency, and CPU utilization across varying queue depths and workload patterns

Apple CoreOS

Software Engineer Intern – Storage Technologies

Cupertino, CA

May 2025 – August 2025

- Enabled user-space testing of kernel storage drivers using a new tool that fakes XNU IOKit driver architecture
- Designed and implemented a fake storage device, automatic driver matching, and a mock infrastructure in C++
- Increased code coverage by over 80% and exposed a 15-year-old XNU kernel bug using the newly built tool

Qumulo

Member of Technical Staff Intern – Cloud Read Cache

Seattle, WA

January 2025 – April 2025

- Cut minimum cluster cost by 33% by designing and deploying single-node clusters with disk-level fault tolerance
- Built automated performance testing for AWS/Azure to benchmark single-node vs. multi-node cluster performance
- Implemented index caching metrics (hits, misses, gap-fills) and I/O tracking to optimize read path efficiency
- Led read cache DKV system upgrade with safe key remapping, quorum barriers, and RPC-based shard coordination

PROJECTS

Sharded Key/Value Service with Paxos Groups (Go)

- Built a fault-tolerant, sharded key/value store in Go with shard migration across Paxos-replicated groups. Implemented a shard master for configuration management and load balancing, atomic two-phase shard handoff during reconfiguration, and duplicate request detection with bounded memory. Ensured linearizability across configuration changes using coordinated log entries.

Multi-threaded Network File Server (C/C++)

- Built a concurrent, crash-consistent network file server in C++ with TCP socket communication, supporting multiple simultaneous users with nested directories. Implemented fine-grained reader-writer locking with hand-over-hand traversal for safe concurrent access, and ensured crash consistency through ordered disk writes that maintain valid metadata invariants.

Virtual Memory Pager (C/C++)

- Designed a demand-paged virtual memory system in C++ supporting multiple processes with swap-backed and file-backed pages. Implemented copy-on-write optimization for fork(), clock page replacement with eager swap reservation, and efficient fault handling with deferred work to minimize disk I/O and page copies.

Thread Library (C/C++)

- Developed a preemptive, kernel-level thread library in C++ supporting multiple CPUs with timer interrupts. Implemented context switching via ucontext, synchronization primitives (mutexes, condition variables) with proper interrupt management, and FIFO scheduling with CPU suspend/resume for efficient resource utilization.